PHP for Beginners — Lesson 2

Nov 8, 2026 · @Jacques MB

By the end of this lesson, learners can build a small multi-page PHP app that remembers the user, stores data in MySQL and uses a simple class.

Lesson overview

Item	Details
Audience	Learners who finished Lesson 1
Duration	About 3 hours (or two 90-minute sessions)
PHP version	PHP 8.x with MySQL or MariaDB (included in XAMPP and Laragon)
Tools	VS Code, XAMPP or Laragon, phpMyAdmin or HeidiSQL

Learning objectives. Learners will be able to:

1. Split a site into reusable parts with `include` and `require`.
2. Remember data between pages with sessions and cookies.
3. Write basic SQL to create, read, update and delete rows.
4. Connect to MySQL with PDO and run prepared statements safely.
5. Create a class, make objects from it and call their methods.

Suggested timing

Part	Minutes
Homework review	10
include and require	15
Sessions and cookies	30
MySQL and SQL basics	25
Break	10
PDO and prepared statements	30
Introduction to OOP	30

Part	Minutes
Mini project	25
Quiz and wrap-up	5

Homework review (Lesson 1 shopping cart). One possible solution to discuss before starting:

```
<?php
function addVat(float $amount, float $rate = 0.18): float
{
    return $amount * (1 + $rate);
}

$cart = [
    ["name" => "Notebook", "price" => 1200, "qty" => 3],
    ["name" => "Pen", "price" => 300, "qty" => 5],
    ["name" => "Bag", "price" => 15000, "qty" => 1],
];

$subtotal = 0;
foreach ($cart as $item) {
    $subtotal += $item["price"] * $item["qty"];
}
echo "Subtotal: $subtotal RWF<br>";            // 20100 RWF
echo "Total with VAT: " . addVat($subtotal) . " RWF"; // 23718 RWF
```

1. Reusing code with include and require

`include` and `require` paste another PHP file into the current one, so a header, menu or footer is written once and used on every page.

Folder structure

```
my-site/
├── includes/
|   ├── header.php
|   └── footer.php
├── index.php
└── about.php
```

includes/header.php

```
<!DOCTYPE html>
<html>
<head>
    <title><?= $pageTitle ?? "My Site" ?></title>
</head>
<body>
    <nav>
        <a href="index.php">Home</a> |
        <a href="about.php">About</a>
    </nav>
```

includes/footer.php

```
    <footer>
        <p>&copy; <?= date("Y") ?> My Site</p>
    </footer>
</body>
</html>
```

index.php

```
<?php
$pageTitle = "Home";
require __DIR__ . "/includes/header.php";
?>
    <h1>Welcome to my site</h1>
<?php require __DIR__ . "/includes/footer.php"; ?>
```

Statement	If the file is missing	Use it for
`include`	Warning, page keeps running	Optional parts (a banner)
`require`	Fatal error, page stops	Essential parts (database config, layout)
`require_once`	Same as `require`, but never loads a file twice	Config files and functions

Teacher tip: `__DIR__` means "the folder this file is in". Using it avoids broken paths when files move into sub-folders.

Practice: create `about.php` that reuses the same header and footer with the title "About".

2. Sessions and cookies

HTTP forgets everything between page loads; sessions keep data on the server and cookies keep small data in the browser, so the site can remember a user.

	Session (`$_SESSION`)	Cookie (`$_COOKIE`)
Stored on	The server	The visitor's browser
Lasts	Until the browser closes or you destroy it	Until the expiry date you set
Can the user see or change it?	No	Yes
Good for	Logged-in user, cart, flash messages	Language or theme preference

Sessions: a simple login

`login.php`

```
<?php
session_start();   // must come before any HTML output

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $username = trim($_POST["username"] ?? "");
    if ($username !== "") {
        $_SESSION["user"] = $username;
        header("Location: dashboard.php");
        exit;
    }
}
?>
<form method="post">
    <input type="text" name="username" placeholder="Your name">
    <button type="submit">Log in</button>
</form>
```

`dashboard.php`

```
<?php
session_start();

if (!isset($_SESSION["user"])) {
    header("Location: login.php");   // not logged in: send back
    exit;
}
?>
<h1>Hello, <?= htmlspecialchars($_SESSION["user"]) ?></h1>
<a href="logout.php">Log out</a>
```

`logout.php`

```
<?php
session_start();
session_destroy();
header("Location: login.php");
exit;
```

Cookies: remember a preference

```
<?php
// Save for 30 days
setcookie("theme", "dark", time() + 60 * 60 * 24 * 30);

// Read it on a later page load
$theme = $_COOKIE["theme"] ?? "light";
echo "Your theme is $theme";

// Delete it: set an expiry in the past
setcookie("theme", "", time() - 3600);
```

Watch out

- `session_start()`, `header()` and `setcookie()` must run before any HTML or `echo`, or you get "headers already sent".
- Always `exit;` after a `header("Location: ...")` redirect.
- Never store passwords or private data in cookies.
- This login is for learning only: a real login checks a password from the database with `password_verify()` (Lesson 3).

3. Databases: MySQL and SQL basics

A database stores data permanently in tables of rows and columns; SQL is the language used to create, read, update and delete (CRUD) that data.

Teacher tip: a table is like an Excel sheet. Each column has a name and type; each row is one record.

Create the database and table (run in phpMyAdmin → SQL tab):

```
CREATE DATABASE school;
USE school;

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    course VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

The four CRUD statements

Action	SQL	Example
Create	`INSERT`	`INSERT INTO students (name, email, course) VALUES ('Eric', 'eric@mail.rw', 'Web Dev');`
Read	`SELECT`	`SELECT name, course FROM students WHERE course = 'Web Dev';`
Update	`UPDATE`	`UPDATE students SET course = 'Networking' WHERE id = 1;`
Delete	`DELETE`	`DELETE FROM students WHERE id = 1;`

Useful SELECT extras

```
SELECT * FROM students ORDER BY name ASC;      -- sort A to Z
SELECT * FROM students LIMIT 10;               -- first 10 rows
SELECT COUNT(*) FROM students;                 -- how many rows
SELECT * FROM students WHERE name LIKE 'A%';   -- names starting with A
```

Watch out: an `UPDATE` or `DELETE` without `WHERE` changes every row in the table. Make learners read their `WHERE` aloud before running it.

Practice: insert three students in phpMyAdmin, then write a `SELECT` that lists them sorted by name.

4. Connecting with PDO and prepared statements

PDO is PHP's built-in way to talk to a database; prepared statements keep user input separate from SQL, which blocks SQL injection.

config/db.php — one connection file, loaded with `require_once`:

```
<?php
$host = "localhost";
$dbname = "school";
$user = "root";
$pass = "";          // XAMPP/Laragon default is an empty password

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4",
        $user,
        $pass,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]
    );
} catch (PDOException $e) {
    die("Database connection failed: " . $e->getMessage());
}
```

Read rows (SELECT)

```
<?php
require_once __DIR__ . "/config/db.php";

$stmt = $pdo->query("SELECT id, name, course FROM students ORDER BY name");
$students = $stmt->fetchAll();

foreach ($students as $s) {
    echo htmlspecialchars($s["name"]) . " — " .
htmlspecialchars($s["course"]) . "<br>";
}
```

Insert with a prepared statement

```
<?php
$stmt = $pdo->prepare(
    "INSERT INTO students (name, email, course) VALUES (:name, :email,
:course)"
);
$stmt->execute([
    "name" => $name,
    "email" => $email,
    "course" => $course,
]);
echo "Saved with id " . $pdo->lastInsertId();
```

Find one row

```
<?php
$stmt = $pdo->prepare("SELECT * FROM students WHERE id = :id");
$stmt->execute(["id" => $id]);
$student = $stmt->fetch();   // false if not found
```

Why prepared statements? Show the danger of joining input into SQL:

```
<?php
// NEVER do this
$sql = "SELECT * FROM students WHERE email = '" . $_GET["email"] . "'";
// A visitor can send:  ' OR '1'='1   and see every student
```

With `prepare()` and placeholders (`:email`), the input is always treated as data, never as SQL.

Rule of thumb: use `query()` only for SQL with no user input; use `prepare()` + `execute()` for everything else.

5. Introduction to OOP

Object-oriented programming groups data (properties) and actions (methods) into a class; a class is the blueprint and each object is one thing built from it.

Teacher tip: a class is the plan for a house; objects are the actual houses. Every house has rooms and doors (properties) and you can open or lock it (methods).

A first class

```php
<?php
class Car
{
    public string $brand;
    public string $color;

    public function honk(): string
    {
        return "{$this->brand} says beep!";
    }
}

$car = new Car();          // create an object
$car->brand = "Toyota";    // set a property
$car->color = "white";
echo $car->honk();         // Toyota says beep!
```

Key words

Term	Meaning
`class`	Defines the blueprint
`new`	Creates an object from the class
`->`	Reaches a property or method of an object
`$this`	"This object", used inside the class
`__construct()`	Runs automatically when the object is created
`public` / `private`	Who can access it: everyone, or only code inside the class

A class with a constructor (PHP 8 constructor promotion):

```php
<?php
class Student
{
    public function __construct(
        public string $name,
        public string $course,
        private array $marks = []
    ) {}

    public function addMark(float $mark): void
    {
```

```
        $this->marks[] = $mark;
    }

    public function average(): float
    {
        if (count($this->marks) === 0) {
            return 0;
        }
        return round(array_sum($this->marks) / count($this->marks), 1);
    }

    public function introduce(): string
    {
        return "I am {$this->name} and I study {$this->course}.";
    }
}

$eric = new Student("Eric", "Web Development");
$eric->addMark(78);
$eric->addMark(85);
$eric->addMark(64);

echo $eric->introduce();              // I am Eric and I study Web
Development.
echo "Average: " . $eric->average();  // Average: 75.7
```

Why `private $marks`? Code outside the class cannot change marks directly (`$eric->marks = ...` gives an error). It must use `addMark()`. This is called **encapsulation**.

Link to Laravel: in Laravel, every model, controller and request is a class. This section is the foundation for that.

Practice: create a `BankAccount` class with a private `$balance`, and methods `deposit()`, `withdraw()` (refuse if not enough money) and `getBalance()`.

6. Mini project: Student register

Learners build one page that adds students to the MySQL `students` table and lists them, using includes, a session flash message, PDO and validation.

What it does

- A form with name, email and course.
- Validates input: name required, email must be valid, email must be unique.
- Saves with a prepared statement, then redirects with a success message stored in the session.

- Lists every student in a table, sorted by name.

Build it in steps

1. Create the `school` database and `students` table (section 3).
2. Create `config/db.php` (section 4).
3. Build the form and the student list (SELECT).
4. Add the INSERT with validation.
5. Add the flash message with `$_SESSION`.

Full solution (`students.php`, next to the `config` folder; tested on PHP 8):

```
<?php
session_start();
require_once __DIR__ . "/config/db.php";

$errors = [];

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $course = trim($_POST["course"] ?? "");

    if ($name === "") {
        $errors[] = "Name is required.";
    }
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Enter a valid email.";
    }

    if (empty($errors)) {
        try {
            $stmt = $pdo->prepare(
                "INSERT INTO students (name, email, course) VALUES (:name,
:email, :course)"
            );
            $stmt->execute(["name" => $name, "email" => $email, "course" =>
$course]);

            $_SESSION["flash"] = "$name was added.";
            header("Location: students.php");   // stops the form re-
submitting on refresh
            exit;
        } catch (PDOException $e) {
            if ($e->getCode() !== "23000") {   // 23000 = duplicate value
                throw $e;
```

```
            }
            $errors[] = "That email is already registered.";
        }
    }
}

$flash = $_SESSION["flash"] ?? null;
unset($_SESSION["flash"]);

$students = $pdo->query("SELECT name, email, course FROM students ORDER BY
name")->fetchAll();
?>
<!DOCTYPE html>
<html>
<head>
    <title>Student Register</title>
</head>
<body>
    <h1>Student Register</h1>

    <?php if ($flash): ?>
        <p style="color: green;"><?= htmlspecialchars($flash) ?></p>
    <?php endif; ?>

    <?php foreach ($errors as $error): ?>
        <p style="color: red;"><?= htmlspecialchars($error) ?></p>
    <?php endforeach; ?>

    <form method="post">
        <p>Name: <input type="text" name="name"></p>
        <p>Email: <input type="email" name="email"></p>
        <p>Course: <input type="text" name="course"></p>
        <button type="submit">Add student</button>
    </form>

    <h2>Registered students (<?= count($students) ?>)</h2>
    <table border="1" cellpadding="6">
        <tr><th>Name</th><th>Email</th><th>Course</th></tr>
        <?php foreach ($students as $s): ?>
            <tr>
                <td><?= htmlspecialchars($s["name"]) ?></td>
                <td><?= htmlspecialchars($s["email"]) ?></td>
                <td><?= htmlspecialchars($s["course"] ?? "") ?></td>
            </tr>
```

```
        <?php endforeach; ?>
    </table>
</body>
</html>
```

New ideas in this project

- `filter_var($email, FILTER_VALIDATE_EMAIL)` checks that an email looks valid.
- **Redirect after POST:** sending the browser back to `students.php` stops a refresh from adding the same student twice.
- **Flash message:** saved in the session, shown once, then removed with `unset()`.
- Error code `23000` means the database refused a duplicate value (the `UNIQUE` email).

Test it: add Eric with eric@mail.rw, then try the same email again ("already registered"), then submit an empty form (two errors).

Extensions for fast learners: add a Delete button per row (a small form posting the `id`, deleted with a prepared `DELETE`); add a search box using `WHERE name LIKE :term`; move the header and footer into `includes/`; style it with Tailwind CSS.

7. Exercises, quiz and homework

Exercises (in class, 3–5 minutes each)

1. Create `includes/menu.php` and include it on two pages.
2. Make a page that counts visits in `$_SESSION["visits"]` and shows "You have visited this page X times".
3. Save the visitor's chosen language (`en` or `rw`) in a cookie and greet them in that language on the next visit.
4. Write a SQL query that lists students in the course "Web Dev", newest first.
5. With PDO, fetch one student by `id` from the URL (`?id=2`) using a prepared statement. Show "Not found" if `fetch()` returns `false`.
6. Create a `Product` class with `name` and `price`, and a method `priceWithVat()` that adds 18%.

Quick quiz

#	Question	Answer
1	What happens if a file loaded with `require` is missing?	A fatal error; the page stops
2	Where is session data stored?	On the server

#	Question	Answer
3	What must be called before using `$_SESSION`?	`session_start()`
4	How do you delete a cookie?	Set it again with an expiry time in the past
5	Which SQL statement reads data?	`SELECT`
6	What is the danger of `DELETE FROM students;`?	It deletes every row (no `WHERE`)
7	What do prepared statements protect against?	SQL injection
8	Which PDO method returns all rows?	`fetchAll()`
9	Which keyword creates an object from a class?	`new`
10	Inside a class, what does `$this` refer to?	The current object

Homework: build a "Library" app with a `books` table (id, title, author, year). One page lists all books; a form adds a book with a prepared statement; a flash message confirms the save. Bonus: a `Book` class with a method `age()` that returns how many years old the book is.

Next lesson: secure login and registration with `password_hash()` and `password_verify()`, updating and deleting records (full CRUD), file uploads, and the MVC pattern — the bridge to Laravel.

Free practice resources: PHP manual: PDO, PHP manual: Sessions, PHP manual: Classes and objects.